

with the snap file system.

```
snapcraft
```

The following command installs the generated local snap:

```
snap install hello_x.xx.xxx.snap
```

Plugins

Plugins help snapcraft in building parts of the snap. For listing all available plugins, use the command given below:

```
snapcraft list-plugins
```

Here are some standard plugins for building snaps:

- **autotools** - to build code based on `./configure`, `make`, `make install`, `cycle`
- **make**, **cmake** - to build `Makefile`, `CMake`
- **qmake** - to build Qt based projects
- **maven**, **gradle** - to build Java code
- **python2**, **python3** - to build Python code or install packages via Pip
- **nodejs** - to build node.js code or install npm packages
- **dump** - this just dumps the contents from the snapcraft directory to the snap file system via staging, prime directories

To get more information on each plugin, use the following command:

```
snapcraft help xxxxx
```

Some snaps are not available in stores but can be installed as custom snaps, by building with snapcraft using suitable plugins. Let's consider a few case studies on building custom snaps.

Node-RED is a visual prototyping tool for wiring services and components together. It can be easily installed through npm as a node.js package. In the *parts* section of *snapcraft.yaml* we'll specify the *node.js* plugin and specify each npm package under node-packages. Here is some code for the *apps* and *parts* of the manifest file:

```
apps:
  red:
    daemon: simple
    command: bin/nodered
    plugs:
      - network-bind
      - network

parts:
  red:
    plugin: nodejs
    node-packages:
      - node-red
```

We can specify other *parts* and *apps* for any Node-RED add-ons required in this snap (Reference: github.com/muka/nodered.snap).

The InfluxData TICK Stack comes with Telegraf, InfluxDb, Chronograf and Kapacitor for data collection, storage, visualisation and analysis with time series data management support. Since binary tar balls are available for all these four components, the dump plugin can be used to build a snap. Here is the code snippet of *snapcraft.yaml* (Reference: github.com/morphis/influxdb-snap):

```
apps:
  influxdb:
    daemon: simple
    command: usr/bin/influxd
    plugs :
      - network-bind
    parts:
      influxdb:
        plugin: dump
        source: https://dl.influxdata.com/influxdb/releases/
        influxdb-<ver>_linux_<arch>.tar.gz
```

IoT gateways like Eclipse Kura as well as servers like Eclipse Leshan and Californium can be packed as snaps using the Maven plugin to build the *parts* and suitable *apps* sections to launch the generated jar files.

Please stay tuned to github.com/rajeshsola/iot-examples/tree/master/snappy-examples for customized *snapcraft.yaml* for the above snaps.

Publishing your snaps

Custom snaps built by you can be published to the store. Just create an account on dashboard.snapcraft.io, log in, create a namespace and publish the snaps with the following commands:

```
snapcraft login
snapcraft register osfy-demos
snapcraft push hello_x.xx.xxx.snap 
```

References

- [1] www.ubuntu.com/core
- [2] www.ubuntu.com/desktop/snappy
- [3] <https://developer.ubuntu.com/core>
- [4] <https://snapcraft.io/docs/reference/plugins/>
- [5] <https://snapcraft.io/docs/core/interfaces>

By: Rajesh Sola

The author is a faculty member of C-DAC's Advanced Computing Training School, and an evangelist in the embedded systems and IoT domains. You can reach him at rajeshsola@gmail.com.